Lab Task-03

Name: Khulood Noor and Baseera Bibi
Registration Number: SP24-BSE-030, SP24-BSE-013
Class: BSE 6A
Subject: Software Quality Engineering

Activity 1 — Review of QuickBite SRS

(QuickBite – Food Delivery App)

Reference: QuickBite SRS (relevant excerpts)

FR1: Users can register using their email address or phone number.

FR2: The system will send an OTP for verification during registration.

FR3: Users can browse nearby restaurants and add menu items to a cart.

FR4: QuickBite will send order status notifications to keep users updated on delivery progress.

FR5: The app allows users to save favorite addresses and reorder previous meals even with a weak or offline connection.

FR6: Users can invite friends to a group order and split the total bill among members.

FR7: QuickBite offers a subscription plan for free delivery and other member perks.

FR8: A user's live order tracking is updated across devices in near real time.

FR9: The first version of QuickBite will be released for Android, with other platforms planned later.

FR10: Users can permanently delete their account and data from the settings menu.

1. Ambiguities:

Ambiguity means a requirement is not clear enough and can be understood in more than one way.

Ambiguity 1 — Registration Verification

Requirement: Users can register using email or phone number, and the system sends an OTP for verification (FR1, FR2).

Problem: It doesn't say how OTP verification would work for someone who signs up using only an email, since OTP codes are normally tied to a phone number.

Identified by Khulood.

Suggestion: The SRS should spell out whether email-only users get their OTP through email, or whether a phone number has to be provided regardless of how someone signs up.

Ambiguity 2 — Order Notifications

Requirement: QuickBite sends order status notifications to keep users updated (FR4).

Problem: There's no mention of how often these go out or at which stages of the order they get triggered.

Identified by Baseera.

Suggestion: Name the actual trigger points — for example, one alert when the order is accepted, one when it's out for delivery, and one on arrival — rather than leaving "notifications" open to interpretation.

Ambiguity 3 — Subscription and Payment

Requirement: QuickBite offers a subscription plan for free delivery and other perks (FR7).

Problem: Nothing in the document says which payment provider will be used or what the subscription actually costs.

Identified by Khulood.

Suggestion: The payment gateway (Google Play Billing, or a card processor such as Stripe) and the exact price of each plan need to be written into the requirement instead of being decided later.

Ambiguity 4 — “Near Real Time” Tracking

Requirement: Order tracking is updated across devices in near real time (FR8).

Problem: “Near real time” isn't something a developer or tester can actually build or check against.

Identified by Baseera.

Suggestion: Replace the vague phrase with a number — for instance, the tracking map should refresh at least every 10 seconds whenever the device has a connection.

2. Missing Requirements:

Missing requirements are important functions or details that are not clearly specified in the SRS.

Missing Requirement 1 — Outdated Cart Items

The SRS lets users reorder favorite meals even offline (FR5), but it doesn't cover what happens if a restaurant's menu or prices changed since that item was saved.

Required: Add a rule for handling outdated cart items, such as flagging price or availability changes before the order is confirmed.

Missing Requirement 2 — Group Order Payment Splitting Logic

The SRS allows group orders and bill splitting (FR6) but doesn't explain how the split is worked out or what happens if a member doesn't pay their share.

Required: Define the splitting method (equal split vs. item-based) and a fallback for when one member's payment fails.

Missing Requirement 3 — Delivery Partner Assignment

The SRS never mentions how a delivery rider gets assigned to an order, or what happens if no rider is available nearby.

Required: Add requirements covering rider-assignment logic and a wait-time or cancellation policy for when no rider accepts the order.

Missing Requirement 4 — Data Retention After Deletion

The SRS allows account deletion (FR10) but doesn't specify what happens to order history, saved cards, or past reviews once the account is gone.

Required: Define what gets deleted right away and what is kept for legal or accounting reasons, and for how long.

Missing Requirement 5 — Accessibility Support

The SRS doesn't mention any accessibility features for users with visual or motor difficulties.

Required: Add requirements for screen-reader support, adjustable text size, and a voice-based ordering option.

3. Inconsistencies:

Inconsistency means that two parts of the SRS do not completely match each other.

Inconsistency 1 — Platform Support

Requirement: One part of the SRS says the first release will be Android-only, with other platforms planned later (FR9).

Problem: Another section describes QuickBite as “available on Android and iOS at launch,” which contradicts FR9.

Correction: Pick one statement and use it everywhere, e.g., “Version 1.0 is Android-only; iOS follows in a later release.”

Inconsistency 2 — Order Status Naming

Requirement: The main order lifecycle is listed as Placed → Preparing → Out for Delivery → Delivered.

Problem: The notifications module also refers to a “Cancelled” status that never shows up in that main lifecycle list.

Correction: Add “Cancelled” to the official list of statuses so every module refers to the same set.

Inconsistency 3 — Subscription Tier Names

Requirement: The overview section names the plans “Free” and “Plus.”

Problem: The pricing page instead lists “Basic,” “Plus,” and “Family” — the names don't line up between the two sections.

Correction: Settle on one naming convention and apply it consistently across the whole document.

Inconsistency 4 — Notification Control

Requirement: One section states users can turn off all notifications from the settings menu.

Problem: Another section says delivery-arrival alerts are mandatory and can't be disabled, which conflicts with the first statement.

Correction: Clarify which notifications, if any, can't be turned off, and keep that consistent in both places.

Homework — Multi-Level Scenario Analysis & Process Transformation

Scenario recap:

EduTech Solutions is a software vendor that builds library and administrative software. They keep having project delays, bugs after release, high staff turnover and the code quality is not consistent between teams.

Task 1 – Defect & Process Identification

#	Observed Practice	Level 1 Symptom
1	Developer implements the client's feature request right away, doesn't check if it clashes with existing code.	Chaotic Process – no procedure before coding starts.
2	PMs guess the timeline by "gut feeling", no past data used.	Chaotic Process – depends on one person's judgement, not repeatable.
3	Every developer has his own folder/naming style, so if he leaves nobody can maintain his code.	Lack of Standardization – classic example, no common convention.
4	Subcontractors hired last minute, no quality check on what they deliver.	Reactive Management – done in a rush, no planning.
5	Testing = quick manual check by developer just before release.	Reactive Management – QA happens too late, after the work is basically done.

Task 2 – Level 2 (Managed) Remediation Plan

Issue #	Identified Failure	Level 2 Process Area	QA Mechanism
1	Direct implementation of requests.	REQM (Requirements Management)	Requirement Change Control form + Traceability Matrix, so every request gets checked before it's built.
2	"Gut feeling" estimation.	PP (Project Planning)	Keep an Estimation Log of old projects and use a simple planning sheet for schedule/effort next time.
3	Inconsistent folder structure.	CM (Configuration Management)	One shared Configuration Management plan – fixed folder layout, naming rule, version control baseline.

Issue #	Identified Failure	Level 2 Process Area	QA Mechanism
4	Subcontractors not vetted.	SAM (Supplier Agreement Management)	Formal agreement with a checklist that has to be signed off before accepting their deliverable.
5	Ad hoc pre-release testing.	PPQA (Process & Product Quality Assurance)	Written test plan + checklist with actual test cases and a defect log before anything is released.